Microservices Architecture

Software Architecture Lecture

July 20, 2026

Learning Outcomes

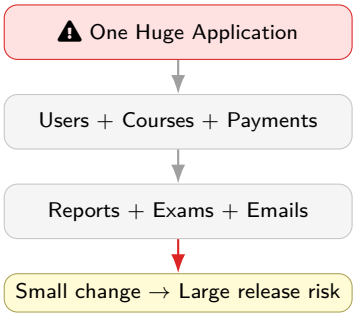
After this lecture, students should be able to:

- Explain the meaning of **microservices architecture**.
- Compare **monolithic**, **modular monolithic**, and **microservices** architectures.
- Describe service communication, API Gateway, database-per-service, and service discovery.
- Explain why Docker and Kubernetes are commonly used with microservices.
- Identify the advantages, challenges, and best practices of microservices.
- Design a simple microservices solution for a real-world application.

Motivation: Why Microservices?

Large applications can become difficult to manage when:

- Many features are added over time.
- Many developers work on the same codebase.
- Different parts need different scaling levels.
- A small change requires a full system release.



What is a Microservice?

Definition

A **microservice** is a small, independently deployable service that implements one business capability and communicates with other services using well-defined APIs.

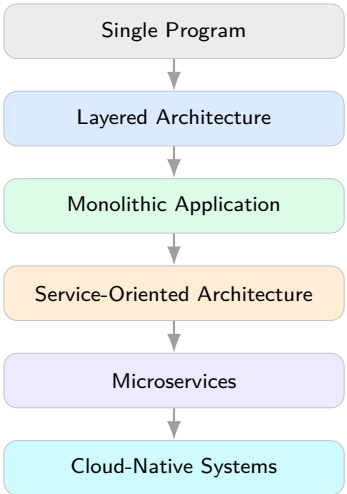
Small

Loosely Coupled

Independently Deployable

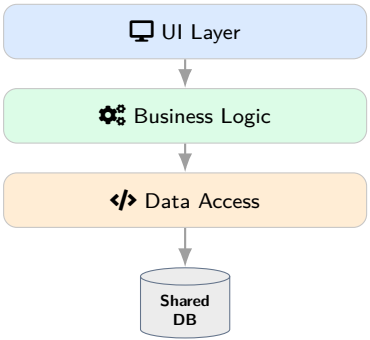
Business Focused

Evolution of Application Architecture



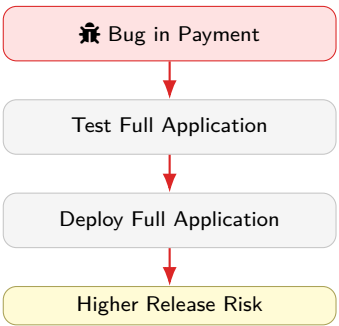
Monolithic Architecture

- One codebase
- One deployment unit
- One shared database in many systems
- Simple to develop at the beginning
- Harder to change and scale as the system grows

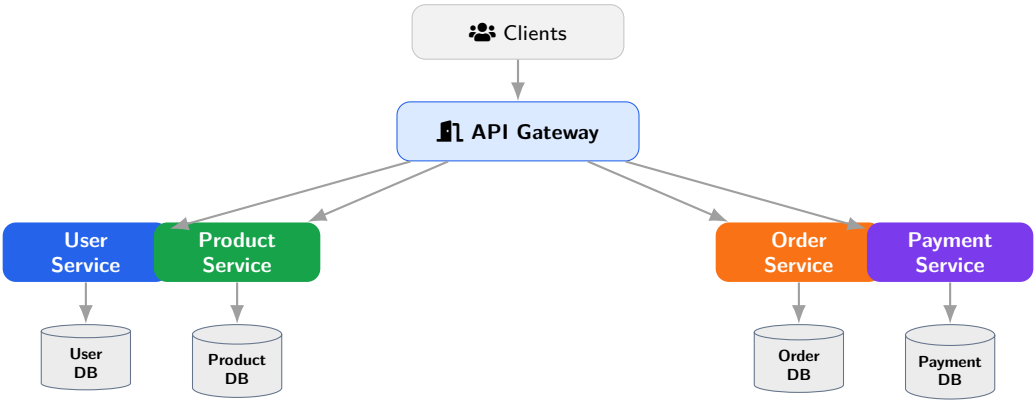


Problems with a Growing Monolith

- Tight coupling between modules
- Long testing and release cycle
- One module failure may affect others
- Difficult to scale only the busy part
- New technology adoption becomes difficult



Microservices Architecture: Big Picture



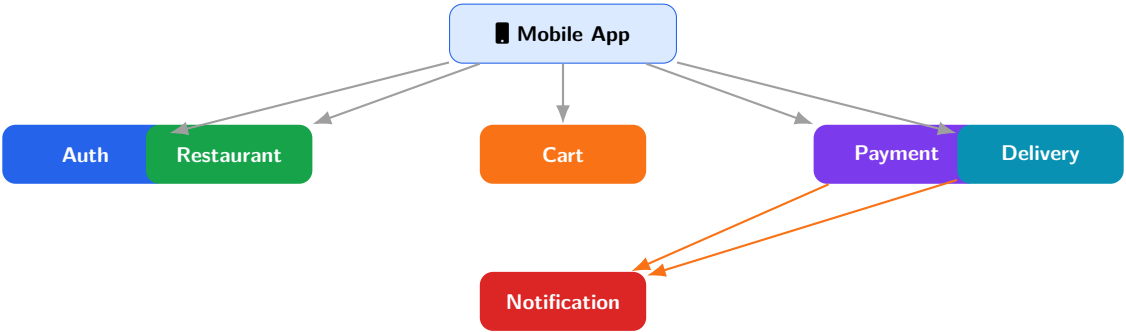
Comparison: Monolithic vs Microservices

Aspect	Monolithic Architecture	Microservices Architecture
Structure	One large application	Many small services
Deployment	Entire system deployed together	Services deployed independently
Scaling	Scale the whole application	Scale only required services
Database	Usually one shared database	Each service may own a database
Technology	Usually one main technology stack	Different services can use different stacks
Failure impact	Failure can affect the whole system	Better fault isolation
Team work	Teams often share one codebase	Teams can own separate services
Complexity	Simpler initially	More operational complexity

Comparison: Modular Monolith vs Microservices

Aspect	Modular Monolith	Microservices
Code organization	Separated into modules within one app	Separated into independently deployed services
Deployment	One deployment unit	Many deployment units
Communication	In-process method calls	Network calls or messages
Data	Often one database, logically separated	Database-per-service is common
Operational cost	Lower	Higher
Good choice when	System is medium-sized or early-stage	System is large, high-scale, multi-team

Real-World Example: Food Delivery System



Key idea: split the system according to business capabilities.

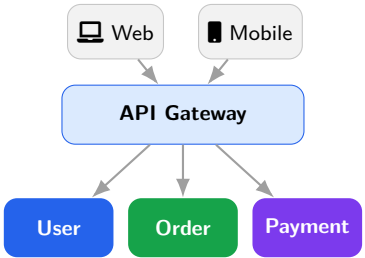
Core Characteristics of Microservices

- **Single Responsibility**: one clear purpose
- **Loose Coupling**: minimal dependency
- **High Cohesion**: related logic together
- **Autonomy**: service can evolve independently
- **Independent Deployment**: release separately
- **Decentralized Data**: service owns its data
- **API Contracts**: clear communication rules
- **Failure Isolation**: failure should be contained

API Gateway

The **API Gateway** is the main entry point for clients.

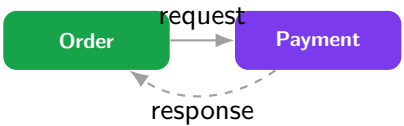
- Request routing
- Authentication
- Rate limiting
- Logging
- Caching
- Response aggregation



Service Communication

Synchronous

- Caller waits for response
- REST or gRPC
- Simple, but can cause delays



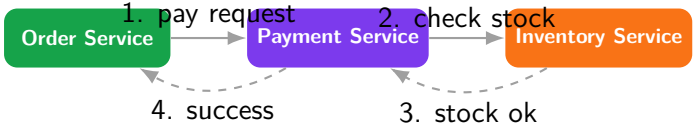
Asynchronous

- Sender publishes message/event
- Kafka or RabbitMQ
- Loosely coupled, but harder to trace

Comparison: REST, gRPC, and Messaging

Aspect	REST	gRPC	Messaging
Style	Request-response	Remote procedure call	Event/message based
Format	Often JSON	Protocol Buffers	JSON, Avro, Protobuf, etc.
Speed	Good	Very fast	Depends on broker
Coupling	Medium	Medium	Lower temporal coupling
Best for	Public APIs, web systems	Internal high-performance APIs	Events, workflows, notifications
Example	HTTP endpoint	Service method call	Kafka topic / RabbitMQ queue

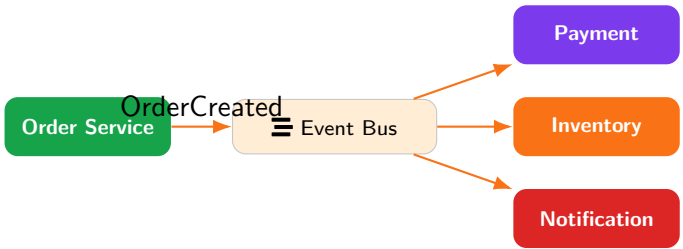
Synchronous Communication Flow



Key Point

Easy to understand, but one slow service can delay the whole user request.

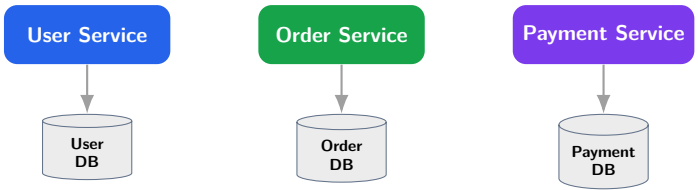
Asynchronous Communication Flow



Key Point

Services react to events without directly depending on each other at request time.

Database per Service



- A service owns its data and controls how others access it.
- Other services should use APIs or events, not direct database access.
- This improves autonomy but makes consistency more difficult.

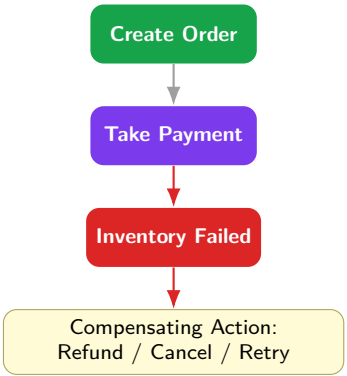
Data Consistency

Problem example:

- Order created
- Payment successful
- Inventory update failed

Possible solution:

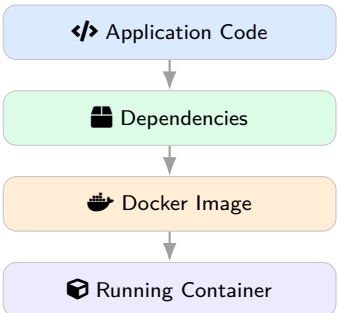
- Retry inventory update
- Cancel order
- Refund payment



Containers and Docker

Containers package the application and dependencies.

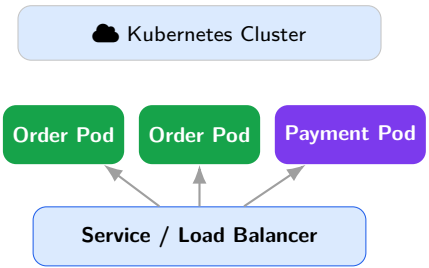
- Same behavior across environments
- Lightweight deployment
- Easy scaling
- Suitable for independent services



Kubernetes for Microservices

Kubernetes helps manage many containers.

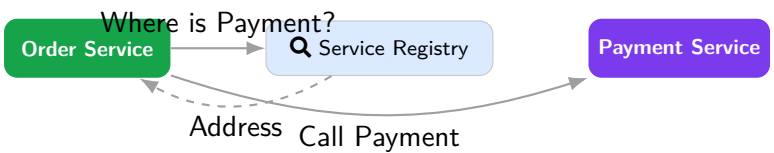
- Deployment
- Scaling
- Load balancing
- Self-healing
- Rolling updates



Service Discovery

Problem

Service instances can start, stop, and move. A service needs a way to find another service.

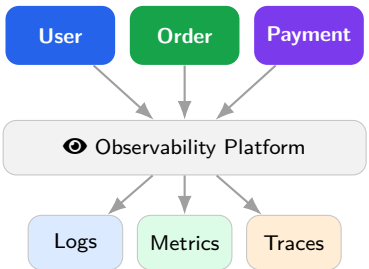


Examples: DNS, Eureka, Consul, Kubernetes Services.

Monitoring, Logging, and Tracing

Microservices need strong observability.

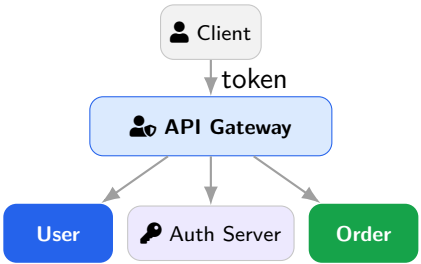
- Centralized logs
- Metrics dashboards
- Distributed tracing
- Alerts



Security in Microservices

Common practices:

- HTTPS everywhere
- JWT / OAuth2
- API Gateway authentication
- Service-to-service authorization
- Secrets management
- Input validation



Comparison: Advantages and Challenges

Advantages	Challenges
Independent deployment	Distributed system complexity
Fine-grained scalability	Network latency and failures
Fault isolation	Data consistency is difficult
Technology flexibility	Monitoring and debugging are harder
Better team autonomy	More DevOps skills are required
Faster release cycles	Testing across services is complex

When Should We Use Microservices?

Use microservices when:

- System is large and growing
- Multiple teams work in parallel
- Independent scaling is needed
- Continuous deployment is important
- DevOps support is available

Avoid microservices when:

- Application is small
- Team is very small
- Requirements are unclear
- Operational skills are limited
- Modular monolith is enough

Best Practices

- Design around business capabilities.
- Keep services small but meaningful.
- Avoid shared databases.
- Define clear API contracts.
- Use API versioning.
- Apply retries and circuit breakers.
- Use centralized logging.
- Automate testing and deployment.
- Monitor service health.
- Start with a modular monolith if uncertain.

Summary

Microservices are:

- Small
- Independent
- Business-focused
- API-based
- Cloud-friendly

But they need:

- Good design
- DevOps practices
- Monitoring and logging
- Security planning
- Careful data management

Microservices solve scaling problems, but introduce distributed-system problems.

Discussion Question

Group Discussion Scenario

A university wants to build a new **Student Management System** including admissions, student profiles, course registration, examination results, payments, notifications, and reports.

Discuss in groups:

1. Would you design it as a monolith, modular monolith, or microservices? Why?
2. What services would you create?
3. Which services should own their databases?
4. Which communication style is suitable: REST, gRPC, or messaging?
5. What risks should the development team consider before choosing microservices?

Tutorial / Exam Questions

1. Define Microservices Architecture with a suitable example.
2. Compare Monolithic and Microservices architectures.
3. Explain the role of an API Gateway.
4. Why is database-per-service recommended in microservices?
5. Compare REST, gRPC, and messaging.
6. Explain the Saga Pattern using an order-payment-inventory example.
7. Discuss the main advantages and challenges of microservices.

Thank You

Questions?